NAME : RIDDHI PATEL

ENROLLMENT: 12402080601115

CLASS : IT-B

SUBJECT : JAVA(2020044502)

BATCH : 2B8

TOPIC : ASSIGNMENT-2

GITHUB LINK :
https://github.com/riddhi280806/java_program_assignment.git

Assignment-2

1.Find prime numbers using multithreading (Thread, Runnable, Executor Framework)

```
import java.util.concurrent.*;

// Runnable implementation
class PrimeTask implements Runnable {
    int start, end;

    PrimeTask(int start, int end) {
        this.start = start;
        this.end = end;
    }

    // Method to check prime
    boolean isPrime(int n) {
        if (n <= 1) return false;
        for (int i = 2; i <= Math.sqrt(n); i++) {
            if (n % i == 0)
                return false;
        }
        return true;
    }

    @Override
    public void run() {
        System.out.println(Thread.currentThread().getName() + " processing range " + start + " to "
+ end);
        for (int i = start; i <= end; i++) {
            if (isPrime(i)) {
                System.out.print(i + " ");
            }
        }
        System.out.println();
    }
}

public class Program1 {
```

```
    public static void main(String[] args) {
        int max = 100;   // limit
        int threads = 4; // number of threads
        int range = max / threads;

        // ===== Using Executor Framework =====
        ExecutorService executor = Executors.newFixedThreadPool(threads);

        int start = 1;

        for (int i = 0; i < threads; i++) {
            int end = start + range - 1;

            // last thread handles remaining numbers
            if (i == threads - 1) {
                end = max;
            }

            PrimeTask task = new PrimeTask(start, end);

            // Submit Runnable task
            executor.execute(task);

            start = end + 1;
        }

        executor.shutdown();
    }
}
```

```
C:\Users\Hetvi Bhatt\OneDrive\Desktop\Sem-4\Java\Assignment-2>java Program1
pool-1-thread-3 processing range 51 to 75
pool-1-thread-2 processing range 26 to 50
pool-1-thread-1 processing range 1 to 25
pool-1-thread-4 processing range 76 to 100
29 31 37 41 43 47
53 59 61 67 71 73
2 3 79 83 89 97
5 7 11 13 17 19 23
```

2.Producer–Consumer problem using synchronization and inter-thread communication

```
class Buffer {
    private int data;
    private boolean hasData = false;

    // Produce method
    public synchronized void produce(int value) {
        try {
            while (hasData) {
                wait(); // wait if buffer is full
            }
            data = value;
            System.out.println("Produced: " + data);
            hasData = true;
            notify(); // notify consumer
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }

    // Consume method
    public synchronized int consume() {
        try {
            while (!hasData) {
                wait(); // wait if buffer is empty
            }
            System.out.println("Consumed: " + data);
            hasData = false;
            notify(); // notify producer
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        return data;
    }
}

// Producer thread
```

```
class Producer extends Thread {
    Buffer buffer;

    Producer(Buffer buffer) {
        this.buffer = buffer;
    }

    public void run() {
        for (int i = 1; i <= 5; i++) {
            buffer.produce(i);
            try {
                Thread.sleep(500);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }
}

// Consumer thread
class Consumer extends Thread {
    Buffer buffer;
    Consumer(Buffer buffer) {
        this.buffer = buffer;
    }
    public void run() {
        for (int i = 1; i <= 5; i++) {
            buffer.consume();
            try {
                Thread.sleep(800);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }
}

// Main class
public class Program2 {
    public static void main(String[] args) {
```

```
        Buffer buffer = new Buffer();

        Producer p = new Producer(buffer);
        Consumer c = new Consumer(buffer);

        p.start();
        c.start();
    }
}
```

```
C:\Users\Hetvi Bhatt\OneDrive\Desktop\Sem-4\Java\Assignment-2>java Program2
Produced: 1
Consumed: 1
Produced: 2
Consumed: 2
Produced: 3
Consumed: 3
Produced: 4
Consumed: 4
Produced: 5
Consumed: 5
```

3.Implement CRUD operations using Collection API (ArrayList, HashMap, TreeMap)

```
import java.util.*;

public class Program3 {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);

      // Collections
      ArrayList<String> list = new ArrayList<>();
      HashMap<Integer, String> map = new HashMap<>();
      TreeMap<Integer, String> tmap = new TreeMap<>();
      int choice;
      do {
         System.out.println("\n--- MENU ---");
         System.out.println("1. ArrayList CRUD");
         System.out.println("2. HashMap CRUD");
         System.out.println("3. TreeMap CRUD");
         System.out.println("4. Exit");
         System.out.print("Enter choice: ");
         choice = sc.nextInt();

         switch (choice) {

            case 1:
               // ArrayList CRUD
               System.out.print("Enter element to add: ");
               String item = sc.next();
               list.add(item);

               System.out.println("List: " + list);

               if (!list.isEmpty()) {
                  System.out.print("Enter index to update: ");
                  int idx = sc.nextInt();
                  System.out.print("Enter new value: ");
                  String val = sc.next();
```

```
            list.set(idx, val);
        }

        System.out.println("After Update: " + list);

        System.out.print("Enter element to delete: ");
        String del = sc.next();
        list.remove(del);

        System.out.println("After Delete: " + list);
        break;

    case 2:
        // HashMap CRUD
        System.out.print("Enter key and value: ");
        int key = sc.nextInt();
        String value = sc.next();
        map.put(key, value);

        System.out.println("Map: " + map);

        System.out.print("Enter key to update: ");
        key = sc.nextInt();
        System.out.print("Enter new value: ");
        value = sc.next();
        map.put(key, value);

        System.out.println("After Update: " + map);

        System.out.print("Enter key to delete: ");
        key = sc.nextInt();
        map.remove(key);

        System.out.println("After Delete: " + map);
        break;

    case 3:
        // TreeMap CRUD
        System.out.print("Enter key and value: ");
        key = sc.nextInt();
```

```
                value = sc.next();
                tmap.put(key, value);
                System.out.println("TreeMap: " + tmap);
                System.out.print("Enter key to update: ");
                key = sc.nextInt();
                System.out.print("Enter new value: ");
                value = sc.next();
                tmap.put(key, value);
                System.out.println("After Update: " + tmap);
                System.out.print("Enter key to delete: ");
                key = sc.nextInt();
                tmap.remove(key);
                System.out.println("After Delete: " + tmap);
                break;
            case 4:
                System.out.println("Exiting...");
                break;
            default:
                System.out.println("Invalid choice");
        }
    } while (choice != 4);
  }
}
```

```
C:\Users\Hetvi Bhatt\OneDrive\Desktop\Sem-4\Java\Assignment-2>java Program3

--- MENU ---
1. ArrayList CRUD
2. HashMap CRUD
3. TreeMap CRUD
4. Exit
Enter choice: 3
Enter key and value: 10
12
TreeMap: {10=12}
Enter key to update: 2
Enter new value: 12
After Update: {2=12, 10=12}
Enter key to delete: 1
After Delete: {2=12, 10=12}

--- MENU ---
1. ArrayList CRUD
2. HashMap CRUD
3. TreeMap CRUD
4. Exit
Enter choice: 4
Exiting...
```

4.Sort Book objects using Comparable and Comparator interfaces

```
import java.util.*;

// Book class implementing Comparable
class Book implements Comparable<Book> {
    int id;
    String name;
    double price;

    // Constructor
    Book(int id, String name, double price) {
        this.id = id;
        this.name = name;
        this.price = price;
    }

    // Comparable (sort by price)
    public int compareTo(Book b) {
        return Double.compare(this.price, b.price);
    }

    void display() {
        System.out.println(id + " " + name + " " + price);
    }
}

// Comparator for sorting by name
class NameComparator implements Comparator<Book> {
    public int compare(Book b1, Book b2) {
        return b1.name.compareTo(b2.name);
    }
}

// Main class
public class Program4 {
    public static void main(String[] args) {

        ArrayList<Book> list = new ArrayList<>();
```

```
        list.add(new Book(1, "Java", 500));
        list.add(new Book(2, "Python", 300));
        list.add(new Book(3, "C++", 400));

        // Sorting using Comparable (by price)
        Collections.sort(list);
        System.out.println("Sorted by Price:");
        for (Book b : list) {
            b.display();
        }

        // Sorting using Comparator (by name)
        Collections.sort(list, new NameComparator());
        System.out.println("\nSorted by Name:");
        for (Book b : list) {
            b.display();
        }
    }
}
```

```
C:\Users\Hetvi Bhatt\OneDrive\Desktop\Sem-4\Java\Assignment-2>javac Program4.java

C:\Users\Hetvi Bhatt\OneDrive\Desktop\Sem-4\Java\Assignment-2>java Program4
Sorted by Price:
2 Python 300.0
3 C++ 400.0
1 Java 500.0

Sorted by Name:
3 C++ 400.0
1 Java 500.0
2 Python 300.0
```

5.Count word occurrences from a βile using File Handling APIs

```
import java.io.*;
import java.util.*;
public class Program5 {
   public static void main(String[] args) {
      HashMap<String, Integer> wordCount = new HashMap<>();
      try {
         // Open file (make sure file.txt exists in same folder)
         BufferedReader br = new BufferedReader(new FileReader("file.txt"));
         String line;
         while ((line = br.readLine()) != null) {
            // Convert to lowercase and split words
            String words[] = line.toLowerCase().split("\\W+");
            for (String word : words) {
               if (word.isEmpty()) continue;
               // Count occurrences
               wordCount.put(word, wordCount.getOrDefault(word, 0) + 1);
            }
         }
         br.close();
         // Display result
         System.out.println("Word Occurrences:");
         for (Map.Entry<String, Integer> entry : wordCount.entrySet()) {
            System.out.println(entry.getKey() + " : " + entry.getValue());
         }
      } catch (IOException e) {
         System.out.println("Error reading file");
      }
   }
}
```

```
C:\Users\Hetvi Bhatt\OneDrive\Desktop\Sem-4\Java\Assignment-2>javac Program5.java

C:\Users\Hetvi Bhatt\OneDrive\Desktop\Sem-4\Java\Assignment-2>java Program5
Word Occurrences:
with : 1
java : 1
programming : 1
```

6.Display all βiles of a given directory using File class

```
import java.io.*;
public class Program6 {
   public static void main(String[] args) {

      // Give directory path (you can change this)
      File dir = new File(".");

      // Get list of files
      File[] files = dir.listFiles();

      if (files == null) {
         System.out.println("Invalid Directory!");
         return;
      }

      System.out.println("Files in Directory:");

      for (File file : files) {
         if (file.isFile()) {
            System.out.println(file.getName());
         }
      }
   }
}
```

```
C:\Users\Hetvi Bhatt\OneDrive\Desktop\Sem-4\Java\Assignment-2>javac Program6.java

C:\Users\Hetvi Bhatt\OneDrive\Desktop\Sem-4\Java\Assignment-2>java Program6
Files in Directory:
Book.class
Buffer.class
Consumer.class
EchoClient.class
EchoClient.java
EchoServer.class
EchoServer.java
NameComparator.class
PrimeTask.class
Producer.class
Program1.class
Program1.java
Program2.class
Program2.java
Program3.class
Program3.java
Program4.class
Program4.java
Program5.class
Program5.java
Program6.class
```

7.Implement TCP Echo Client–Server program

```
import java.io.*;
import java.net.*;

public class EchoServer {
   public static void main(String[] args) {
      try {
         ServerSocket server = new ServerSocket(5000)
         System.out.println("Server started... Waiting for client");
         Socket socket = server.accept();
         System.out.println("Client connected");
         BufferedReader in = new BufferedReader(
               new InputStreamReader(socket.getInputStream()));
         PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
         String message;
         while ((message = in.readLine()) != null) {
            System.out.println("Client: " + message);
            out.println("Echo: " + message); // send back same message

            if (message.equalsIgnoreCase("exit")) {
               break;
            }
         }
         socket.close();
         server.close();
         System.out.println("Server closed");
      } catch (IOException e) {
         e.printStackTrace();
      }
   }
}
```

```
C:\Users\Hetvi Bhatt\OneDrive\Desktop\Sem-4\Java\Assignment-2>javac EchoServer.java

C:\Users\Hetvi Bhatt\OneDrive\Desktop\Sem-4\Java\Assignment-2>javac EchoClient.java

C:\Users\Hetvi Bhatt\OneDrive\Desktop\Sem-4\Java\Assignment-2>java EchoServer
Server started... Waiting for client
Client connected
Client: hello world!!!!
Client: This java Program
Client: exit
Server closed
```

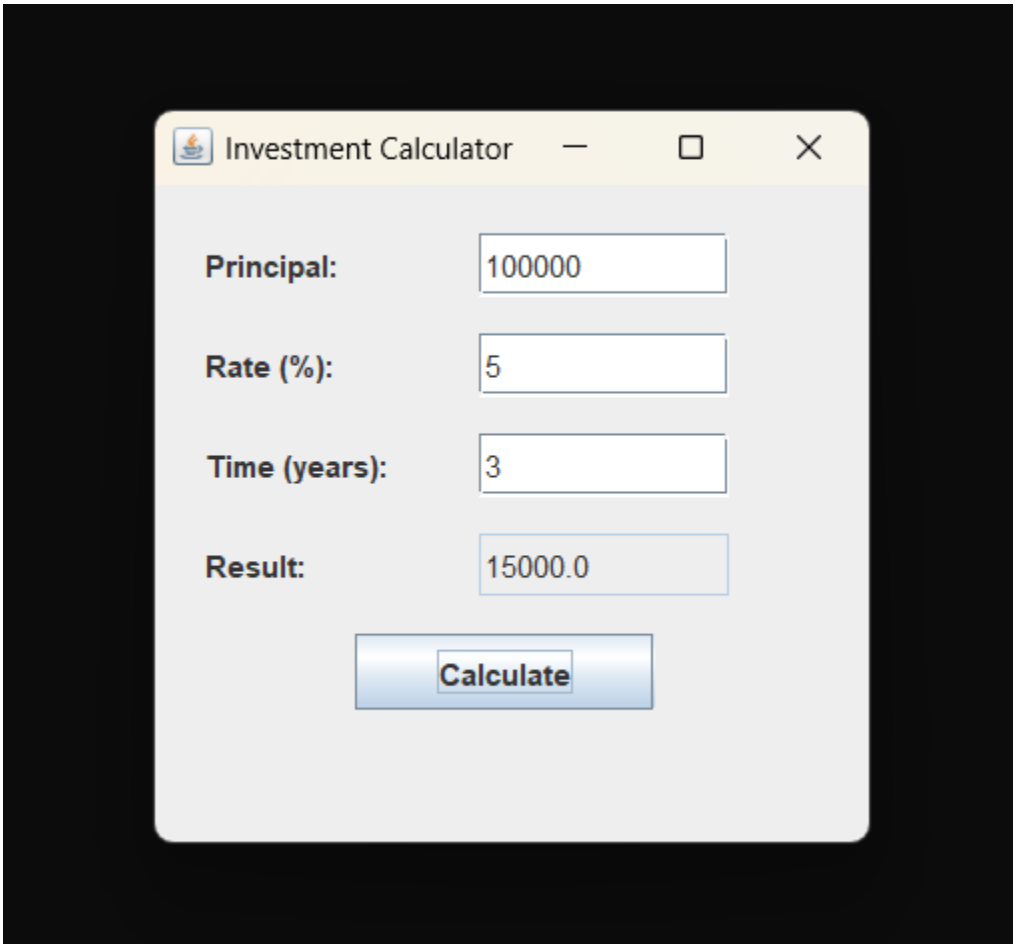
```
import java.io.*;
import java.net.*;
import java.util.*;

public class EchoClient {
   public static void main(String[] args) {
      try {
         Socket socket = new Socket("localhost", 5000);

         BufferedReader in = new BufferedReader(
               new InputStreamReader(socket.getInputStream()));
         PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
         Scanner sc = new Scanner(System.in);

         String message;

         while (true) {
            System.out.print("Enter message: ");
            message = sc.nextLine();
            out.println(message); // send to server
            String response = in.readLine();
            System.out.println("Server: " + response);

            if (message.equalsIgnoreCase("exit")) {
               break;
            }
         }
         socket.close();
      } catch (IOException e) {
         e.printStackTrace();
      }
   }
}
```

```
C:\Users\Hetvi Bhatt>cd C:\Users\Hetvi Bhatt\OneDrive\Desktop\Sem-4\Java\Assignment-2

C:\Users\Hetvi Bhatt\OneDrive\Desktop\Sem-4\Java\Assignment-2> java EchoClient
Enter message: hello world!!!!
Server: Echo: hello world!!!!
Enter message: This java Program
Server: Echo: This java Program
Enter message: exit
Server: Echo: exit
```

8.Develop GUI-based Investment Calculator using Swing

```
import javax.swing.*;
import java.awt.event.*;

public class Program8 extends JFrame implements ActionListener {

    JLabel l1, l2, l3, l4;
    JTextField t1, t2, t3, t4;
    JButton btn;

    Program8() {
        setTitle("Investment Calculator");
        setSize(300, 300);
        setLayout(null);

        // Labels
        l1 = new JLabel("Principal:");
        l2 = new JLabel("Rate (%):");
        l3 = new JLabel("Time (years):");
        l4 = new JLabel("Result:");

        // TextFields
        t1 = new JTextField();
        t2 = new JTextField();
        t3 = new JTextField();
        t4 = new JTextField();
        t4.setEditable(false);

        // Button
        btn = new JButton("Calculate");
        btn.addActionListener(this);

        // Set positions
        l1.setBounds(20, 20, 100, 25);
        t1.setBounds(130, 20, 100, 25);

        l2.setBounds(20, 60, 100, 25);
        t2.setBounds(130, 60, 100, 25);

        l3.setBounds(20, 100, 100, 25);
```

```
        t3.setBounds(130, 100, 100, 25);

        l4.setBounds(20, 140, 100, 25);
        t4.setBounds(130, 140, 100, 25);

        btn.setBounds(80, 180, 120, 30);

        // Add components
        add(l1); add(t1);
        add(l2); add(t2);
        add(l3); add(t3);
        add(l4); add(t4);
        add(btn);

        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setVisible(true);
    }

    public void actionPerformed(ActionEvent e) {
        try {
            double p = Double.parseDouble(t1.getText());
            double r = Double.parseDouble(t2.getText());
            double t = Double.parseDouble(t3.getText());

            // Simple Interest Calculation
            double si = (p * r * t) / 100;

            t4.setText(String.valueOf(si));

        } catch (Exception ex) {
            JOptionPane.showMessageDialog(this, "Enter valid numbers!");
        }
    }

    public static void main(String[] args) {
        new Program8();
    }
}
```


Investment Calculator
Principal:
100000
Rate (%):
5
Time (years):
3
Result:
15000.0
Calculate